Comic Dialogue Text Analyser: A Compiler-Style Front-End for Lexical Analysis, Speech-Bubble Decomposition and Character Prominence Modelling in Post-OCR Comic Dialogue

Kartikeya Behl (24BAI0083), Samarth Pritmani (24BAI0078), Vayun Tejwani (24BAI0027)
Compiler Design Laboratory | Team 11 | Slot G2+TG2

Abstract—Comic books and graphic novels are digitised by detecting speech balloons and passing them through Optical Character Recognition (OCR). The extracted text is noisy: it is set in capitals, carries expressive punctuation such as '...' and '?!', contains sound effects such as 'BOOM' and 'AAARGH', and is littered with stray scan symbols such as '~' and '#'. Strict Context-Free Grammar (CFG) parsers can reject 60–75% of such dialogue because it consists largely of fragments, shouts and informal constructions. This paper presents the Comic Dialogue Text Analyser, a compiler-style front-end that continues where a bubble-extraction and OCR pipeline stops. It combines a speaker splitter and text normaliser, a speech-bubble clause splitter, a comic-specific lexer with an OCR error tracker and a symbol table, a rule-based sentence classifier and tone detector, and a Character Popularity Engine that detects direct address (vocatives), resolves aliases through swappable JSON configurations and scores characters with a weighted lexical model. Vocatives also define a directed conversational network (A → B) that is analysed with degree and betweenness centrality and PageRank. Results are exported as LaTeX tables, Excel workbooks, CSV leaderboards and JSON logs. On benchmark dialogues and on 6,335 lines from 18 superhero screenplays of the Cornell Movie-Dialogs Corpus, the system reached a 99.8% processing success rate against 38.5% for a CFG-parser baseline, at an average of 0.098 ms per sentence, which is 187 times faster than the baseline.

***Index Terms*—Comic dialogue, lexical analysis, compiler front-end, symbol table, speech-bubble decomposition, vocative addressing, conversational social networks, betweenness centrality, character popularity modelling, OCR error detection.**

I. INTRODUCTION

Comics and graphic novels are a multimodal storytelling medium in which narrative progression depends on the close coordination of illustrated panels and textual speech balloons. As digital reading platforms spread, digitising comic heritage and enabling accessibility services such as text-to-speech, automated translation, semantic script search and narrative indexing has become an active research area. A standard digitisation workflow detects the page panels, segments the speech balloons and applies OCR to obtain the raw dialogue strings [1]–[3]. The base paper of this work [4] follows exactly this route and ends with a refined text string for every detected balloon.

Downstream language analysis of such post-OCR dialogue meets obstacles that stem from comic typography and colloquial speech:

- **All-caps lettering** defeats capitalisation-based Named Entity Recognition.
- **Expressive punctuation** such as trailing ellipses ('...'), interrobangs ('?!') and repeated exclamation marks ('!!!') carries hesitation, astonishment and urgency.
- **Onomatopoeia and letter lengthening** ('POW', 'THWIP', 'NOOOOO', 'AAARGH') is absent from conventional lexicons.
- **Multi-sentence balloons** blend commands, questions and hesitant statements, e.g. 'LOOK OUT, BRUCE! WHERE DID HE GO? HE WAS JUST HERE...'.
- **OCR scan artefacts** such as '~', '#' and '|' enter the transcript from creases and balloon borders.

Early approaches parsed dialogue with strict Context-Free Grammars on the assumption that speech can be derived from formal sentence structures. Spontaneous dialogue and comic shouting, however, are full of fragments, one-word exclamations and inverted phrasing, and CFG parsers can reject between 60% and 75% of valid lines. Informal discourse is known to make CFG parsers fail badly [5], [6].

This work discards the brittle syntax tree. Treating noisy dialogue the way a compiler treats source code, we build a fixed sequence of deterministic phases (normalisation, clause splitting, lexical analysis, classification, vocative detection, prominence scoring and network analysis) in which no phase aborts on malformed input. The main contributions, detailed in Section X, are: (i) a comic-specific lexer with nine token classes, an OCR error tracker and a symbol table; (ii) a clause splitter that decomposes multi-sentence balloons while protecting abbreviations; (iii) a lexical vocative detector with JSON-driven alias resolution and a weighted prominence score; (iv) a directed conversational network with centrality analysis; and (v) an export suite and an evaluation against a CFG-parser baseline on 6,335 screenplay lines.

The rest of the paper is organised as follows. Section II reviews related work. Section III describes the base pipeline and formulates the problem. Section IV presents the methodology. Section V covers implementation and experimental setup, Section VI reports results, Section VII summarises project outcomes, Section VIII discusses limitations, Section IX concludes, and Section X lists the contributions made over the base paper.

II. LITERATURE SURVEY

A. Comic Digitisation, Balloon Detection and OCR

Early work on comic digitisation segmented panels using gutter geometry [7]. Augereau et al. [1] and Matsui et al. [8] surveyed comic and manga analysis and noted that speech balloons vary drastically in shape, which complicates automatic text extraction. Dubray and Laubrock [2] used deep convolutional networks to detect and segment balloons across diverse comic styles, and Rigaud et al. [9], [10] used active contours and typographic features to extract text lines. Lenadora et al. [3] combined bubble extraction with Tesseract OCR and observed that the output frequently contains detached punctuation and stray border marks. Custom comic fonts are reported to push OCR character error rates to roughly 15–22% [11], [12], so a dialogue-processing pipeline has to absorb lexical noise without halting.

B. Compiler Front-Ends and Lexical Processing of Informal Text

Classical compiler design divides translation into linear phases: scanning, symbol-table management, parsing and semantic checking [13], [14]. Applying compiler techniques to natural language has been discussed in [15], [16], but general parsers such as Earley and CYK [17], [18] enforce rigid derivations. Informal discourse is known to cause CFG parsers to fail badly [5], [6]; Foster [19] and Geman and Johnson [20] argued for shallow, deterministic pattern matching over all-or-nothing grammar trees, and Gimpel et al. [21] showed that character lengthening and repeated punctuation behave as first-class lexical tokens. Following finite-state tokenisation principles [22], [23], our pipeline uses deterministic lexical automata that run in linear time.

C. Character Networks and Prominence in Scripts

Elson et al. [24] extracted conversational networks from nineteenth-century novels, and Danescu-Niculescu-Mizil and Lee [25] introduced the Cornell Movie-Dialogs Corpus of more than 304,000 utterances, showing that conversational dynamics reflect character status. Agarwal et al. [26] analysed screenplay interactions by who addresses whom. Alias resolution is necessary because entities are invoked by titles, alter egos and nicknames [27], [28]. Labatut and Bost [29] surveyed narrative networks, indicating that interaction frequency can reflect narrative importance better than raw dialogue volume. Studies of graphic-novel discourse [30], [31] point to direct address (‘Look out, Batman!’) as an indicator of conversational focus, and graph centrality measures have been used to identify core protagonists [32], [33].

D. Research Gap

The literature on balloon detection ends at OCR text, while the literature on character networks assumes clean script text. What is missing is a robust, fast and transparent stage in between that turns noisy OCR dialogue into structured clauses, tokens, error records, addressees and character statistics. This paper fills that gap.

III. BASE PIPELINE AND PROBLEM FORMULATION

A. Base Pipeline

The base paper [4] digitises comic pages in five modules. *Panel extraction* converts the page to grayscale, thresholds it and extracts panel masks from connected components and contours. *Feature extraction* finds the contour list of each panel. *Bubble localisation* outlines candidate balloons and removes oversized candidates that contain smaller ones. *Speech bubble extraction* crops every localised balloon. Finally, an *innovation module* resizes and denoises each crop and reads it with the LSTM engine of Tesseract [34] through pytesseract, then separates junk characters from the script and applies a spell check. Its reported evaluation counts correctly detected balloons (for example 3 of 4, an accuracy of 0.75, on one sample panel).

B. Remaining Gap

The refined output of the base pipeline is a flat string per balloon, for example ‘WHERE COULD HE BE, NATASHA? WHY ARE YOU SO WORRIED, TONY? CAPTAIN AMERICA IS MORE THAN ABLE TO TAKE CARE OF HIMSELF.’ Several communicative acts and possibly several speakers are merged in one string, removed junk is returned only as an unlocated string, and nothing states who is speaking, who is addressed, what kind of sentence each clause is, or how often each character matters.

C. Problem Formulation

Let $L = (l_1, \ldots, l_n)$ be a sequence of dialogue lines, each optionally prefixed by a speaker tag. The task is to produce, for every line, a speaker, a list of clauses with sentence type and tone, a token stream and a list of located lexical errors, and, for the whole corpus, a symbol table, a ranked list of canonical characters, and a directed weighted graph $G = (V, E)$ of who addresses whom with its centrality metrics. The solution must never abort on malformed input and must run in time linear in the input length.

IV. PROPOSED METHODOLOGY

A. Overview and Design Principles

The methodology treats noisy comic dialogue the way a compiler treats source code: text passes through a fixed sequence of deterministic phases, each consuming the output of the previous one and none aborting on malformed input. Instead of building a CFG parse tree, every phase relies on finite-state style pattern matching, so each line is processed in linear $O(N)$ time. The three design principles are *determinism* (ordered rules whose precedence resolves every ambiguity), *non-aborting error handling* (noise is logged, not raised) and *configurability* (lexicons, abbreviations, weights and alias files are data, not code). Fig. 1 shows the resulting architecture of seven modules: six analysis modules plus the export engine.

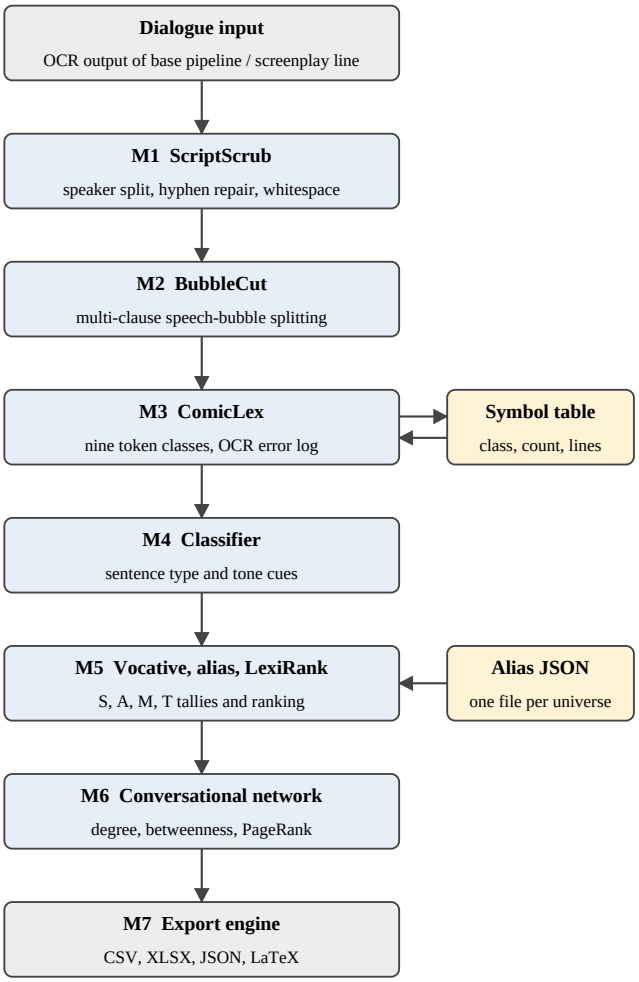


Fig. 1. Architecture of the Comic Dialogue Text Analyser. Modules are numbered in execution order.

B. Module M1: Speaker Split and Text Normalisation (ScriptScrub)

Raw lines from OCR output or a screenplay dataset, written as ‘SPEAKER: dialogue’ or ‘[SPEAKER]: dialogue’, are divided into a speaker tag and a dialogue body by one anchored regular expression. The body is cleaned by repairing words broken across lines by hyphenation (‘SEC- TOR’ becomes ‘SECTOR’) and by collapsing irregular whitespace. A line without a speaker prefix is kept as anonymous dialogue instead of raising an error (Algorithm 1).

Algorithm 1. ScriptScrub: speaker split and normalisation

```
Input : raw line L          Output: (speaker S, text D)
1  m <- Match(SPK_RE, L)
2  if m then S <- Trim(m.b_spk or m.spk); T <- m.diag
3  else      S <- null;                  T <- L
4  D <- Replace(T, HYPH_RE, '\1\2')   // repair SEC- TOR
5  D <- Replace(D, '[ \t]+', ' ')      // collapse blanks
6  return (S, Trim(D))
SPK_RE  = ^\s*(?:\[(?P<b_spk>[^\]]+)\]|
          (?P<spk>[A-Za-z0-9_\-\.\s]+?))\s*:\s*(?P<diag>.*)$
HYPH_RE = (\b[A-Za-z]+)-\s+([A-Za-z]+\b)
```

C. Module M2: Multi-Clause Speech-Bubble Splitting (BubbleCut)

A single balloon often holds several communicative acts. Analysing it as one unit makes a shout or command hide beneath a trailing question and limits vocative extraction. Before lexing, each balloon is therefore divided at the comic boundary terminators ‘!’, ‘?’, ‘?!’, ‘...’ and ‘.’. A look-ahead condition requires that a new clause (a capital letter, digit, quote or bracket) follow the terminator, and a protected-abbreviation list (DR., MR., CAPT., O.S., V.O., ETC., VS.) prevents false splits. Every clause is classified and searched for vocatives independently, but the results are credited back to the same speaker so that spoken-line counts are not inflated (Algorithm 2).

Algorithm 2. BubbleCut: boundary splitting

```
Input : cleaned string D, abbreviation set ABB
Output: list of clauses C
1  C <- []; last <- 0
2  for each match m of BOUNDARY_RE in D do
3    if m.term = '.' and PrecedingWord(D, m) in ABB then
4      continue                  // protect DR., CAPT. ...
5    clause <- Trim(D[last .. m.end])
6    if clause is not empty then Append(C, clause)
7    last <- m.end
8  if last < Length(D) then Append(C, Trim(D[last .. ]))
9  return C
BOUNDARY_RE = (\.\.\.+|…|(?:\?+!+|!+\?+)[\?!]*|!+|\?+|\.)
              (?=\s+[A-Z0-9"'‘“(\[\-]|$)
```

D. Module M3: Comic Lexer, OCR Error Tracker and Symbol Table (ComicLex)

The cleaned clause is scanned once, left to right, by an ordered set of regular-expression rules whose precedence resolves every ambiguity (Table I). Each lexeme becomes one of nine token classes. Elongated shouts such as ‘AAARGH’ are caught by a repeated-letter rule (a letter repeated three or more times) and onomatopoeia by a sound-effect lexicon. A character matching no rule, such as the scan artefacts ‘~’ and ‘#’, is logged as a lexical error with its character offset, an UNKNOWN token is emitted, and scanning continues, so OCR noise is tracked without halting the batch (Algorithm 3). Every token is entered in a hash-based symbol table keyed by the upper-case lexeme that stores its class, frequency, line references and a proper-name flag.

TABLE I
TOKEN CLASSES OF COMICLEX IN RULE ORDER

Class	Examples	Rule
ELLIPSIS	..., …	three or more dots, or the … glyph
INTERROBANG	?!, !?, ?!!	mixed run of ? and !
CONTRACTION	DON'T, I’M	letters, apostrophe, letters
WORD / SOUND_FX	BATMAN / BOOM, AAARGH	letters; SOUND_FX if in the lexicon or a letter repeats 3+ times
NUMBER	10, 99	digit run
TERMINATOR	. ? !	single sentence-end mark
PUNCT	, : ; -	separator marks
UNKNOWN	~ # |	matches no rule; logged as lexical error with offset

Algorithm 3. ComicLex: tokenisation and error logging

```
Input : cleaned string D, symbol table T
Output: Tokens, Errors
1  i <- 0; n <- Length(D)
2  while i < n do
3    if D[i] is whitespace then i <- i+1; continue
4    s <- D[i .. n]
5    if   Match('^(\.\.\.+|…)', s)           -> ELLIPSIS
6    elif Match('^(\?+!+|!+\?+)[\?!]*', s)    -> INTERROBANG
7    elif Match("^[A-Za-z]+['’][A-Za-z]+", s) -> CONTRACTION
8    elif m <- Match('^[A-Za-z]+', s) then
9      SOUND_FX if IsSoundFX(m) or Repeats3(m) else WORD
10   elif Match('^[0-9]+', s)                 -> NUMBER
11   elif Match('^[.?!]', s)                  -> TERMINATOR
12   elif Match('^[,:;\-]', s)                -> PUNCT
13   else Log(Errors, symbol = D[i], offset = i)
14        token <- UNKNOWN(D[i])
15   Add(Tokens, token); InsertIntoSymbolTable(T, token)
16   i <- i + Length(token)
17 return (Tokens, Errors)
```

E. Module M4: Rule-Based Sentence Classification and Tone Detection

A priority-ordered rule cascade, evaluated first-match-wins in the order sound-effect-only, interrobang, question mark, question starter, imperative verb, exclamation and default statement, assigns each clause one of four types from its opening word and closing token. A **Question** closes with ‘?’ or ‘?!’ and opens with a Wh-word (WHERE, WHO, HOW) or an auxiliary (COULD, CAN, DID, WILL). A **Command** opens with an imperative verb (LOOK, STOP, RUN, WAIT, LISTEN) and has no explicit subject pronoun. An **Exclamation** ends with ‘!’ or consists mainly of sound effects. Everything else is a **Statement**. Tone cues are extracted in parallel from the token classes: shouting from ‘!’ and capitals, hesitation from ‘...’, surprise or incredulity from ‘?!’, and action or pain from sound effects.

F. Module M5: Vocatives, Mentions, Aliases and Character Prominence (LexiRank)

Lexical patterns replace the CFG vocative rule. A *leading* (NAME + comma), *trailing* (comma + NAME + terminator) or *medial* (comma + NAME + comma) pattern identifies the character who is directly addressed, for instance ‘WHERE COULD HE BE, NATASHA?’. Other known names are counted as third-person mentions. Multi-word names (BLACK WIDOW) are matched with three-word and two-word windows, and all variants (BRUCE, THE DARK KNIGHT, THE BAT) are merged into one canonical identity (BATMAN) through an external JSON alias file that can be swapped for each fictional universe.

For every canonical character the engine tallies lines spoken S, times addressed A, third-person mentions M and high-tone lines T, and combines them as

$$\text{Popularity}(C) = w_s \cdot S + w_a \cdot A + w_m \cdot M + w_t \cdot T \quad (1)$$

with $w_s = 1.0$, $w_a = 1.5$, $w_m = 0.8$ and $w_t = 0.2$. Being addressed carries the highest weight because a direct vocative shows that the conversation is focused on that character. The name

'LexiRank' denotes this multi-factor weighted lexical tally; it is not the graph-based LexRank summarisation algorithm [35], and the graph-based analysis is performed separately in M6 (Algorithm 4).

Algorithm 4. LexiRank: prominence scoring

```
Input : speaker S, clause tokens K, alias map A, weights W
Output: characters ranked by score
1  Add <- DetectVocative(K)   // 'NAME,' ', NAME?' ', NAME,'
2  Men <- FindMentions(K, exclude = {S, Add})
3  if S is not null then
4    c <- Resolve(A, S);  Tally[c].S += 1
5    if HasHighTone(K) then Tally[c].T += 1
6  if Add is not null then Tally[Resolve(A, Add)].A += 1
7  for m in Men do Tally[Resolve(A, m)].M += 1
8  for each character c in Tally do
9    Score[c] <- ws*S + wa*A + wm*M + wt*T
10 return SortDescending(Score)
```

G. Module M6: Conversational Network and Centrality Analysis

Every vocative adds weight to a directed edge $u \rightarrow v$ of the interaction graph $G = (V, E)$, where V is the set of canonical characters and the weight $w(u, v)$ counts how often u addresses v. The following metrics check whether the weighted score of Eq. (1) agrees with each character's structural role.

- **In-degree** (conversational reception, the focal point of other characters' dialogue) and **out-degree** (conversational initiation); **degree centrality** combines both.
- **Betweenness centrality** $C_B(v) = \Sigma_{s \neq v \neq t} \sigma_{st}(v)/\sigma_{st}$, the share of shortest conversational paths between other characters that pass through v [36], [37]; it reveals narrative brokers who connect disparate character factions.
- **PageRank** [38], a recursive measure of how often a character is addressed by characters who are themselves frequently addressed.

The metrics are computed with NetworkX [39]; a native fallback provides the degree metrics when the library is unavailable.

H. Module M7: Reporting, Export and Evaluation Protocol

Statistics, leaderboards and network metrics are exported as CSV files (leaderboard and edge list), a multi-sheet Excel workbook (popularity leaderboard, network centrality, distributions, lexical OCR errors), a full JSON log and booktabs LaTeX tables for direct inclusion in papers. Evaluation uses three measures: *processing success rate* (share of lines that complete without an unhandled failure), *average sentence latency* (total wall time divided by the number of sentences) and *throughput* (sentences per second). Because every phase is a single left-to-right scan, the cost per line is linear in its length, in contrast to the cubic cost of CYK or Earley parsing [17], [18].

V. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. Implementation

The system is implemented in Python as a modular package whose files map one-to-one onto the modules of Fig. 1 (Table II). A *ComicDialoguePipeline* class exposes *analyze_sentence*, *analyze_bubble* and *analyze_batch*. A command-line interface supports single-sentence, batch-file, compact, no-bubble-split, alias-file and export-directory modes, and an interactive Google Colab notebook runs every module live. Alias presets (*aliases.json* and *aliases_manga.json*) demonstrate the swappable-universe design, and an interactive Plotly graph visualises the network.

TABLE II
MAPPING OF MODULES TO SOURCE FILES

Module	Source files
M1 ScriptScrub	cleaner.py
M2 BubbleCut	bubble_splitter.py
M3 ComicLex	lexer.py, symbol_table.py, config.py
M4 Classifier	classifier.py
M5 Vocative, alias, LexiRank	mention_finder.py, alias_loader.py, character_popularity.py, reporter.py
M6 Network	network_analyzer.py
M7 Export	export_engine.py, main.py

B. Datasets

Two datasets are used (Table III). The *benchmark set* contains 18 hand-prepared lines (28 clauses, 12 characters) that exercise OCR noise ('~', '#'), sound effects and multi-sentence balloons. The *Cornell Superhero Corpus* consists of 6,335 dialogue turns from 18 superhero screenplays (Batman, Superman, Spider-Man, Blade and X-Men scripts) of the Cornell Movie-Dialogs Corpus [25], converted by a preprocessor into the 'Speaker: line' format; it contains 201 distinct characters.

C. Baseline and Metrics

The proposed front-end is compared with a traditional CFG-parser baseline on the 6,335-sentence corpus using processing success rate, average sentence latency, throughput and grammar-coverage failure rate (Section IV-H).

TABLE III
DATASET CHARACTERISTICS

	Benchmark set	Cornell Superhero Corpus
Source	Hand-prepared test cases	Cornell Movie-Dialogs (18 screenplays)
Size	18 lines / 28 clauses	6,335 dialogue turns
Distinct characters	12	201
Key traits	OCR noise (~, #), sound FX, multi-sentence bubbles	Batman, Superman, Spider-Man, Blade, X-Men scripts

VI. RESULTS AND DISCUSSION

A. Bubble Splitting and OCR Noise Interception

On the multi-sentence benchmark balloon 'LOOK OUT, BRUCE! WHERE DID HE GO? HE WAS JUST HERE...', BubbleCut produced three clauses: (1) 'LOOK OUT, BRUCE!', classified as a Command with shouting tone and BRUCE addressed; (2) 'WHERE DID HE GO?', a Question with neutral tone; and (3) 'HE WAS JUST HERE...', a Statement with hesitation tone. Splitting thus exposes the command that a whole-balloon classifier would have hidden beneath the question. In the noisy line 'BATMAN~ WE HAVE A 10-99 AT SECTOR 4#.' the stray symbols '~' and '#' were intercepted and logged with exact character offsets while analysis of the line continued.

B. Character Prominence

Table IV lists the top characters of the Cornell Superhero Corpus under Eq. (1), and Fig. 2 breaks the scores of the twelve most prominent characters into their four components. Lines spoken dominate the score of every character: for BATMAN they supply 382 of 475.2 points (80.4%), with 45.0 points from being addressed 30 times and 46.4 from mentions. Among the entries of Table IV, BATMAN and SUPERMAN are the only ones with substantial addressed counts (30 each), which lifts them clearly above KARL and EMMA, whose scores come almost entirely from their own lines. This is the behaviour the weight on A is meant to produce: volume alone does not make a character the focus of the conversation.

TABLE IV
TOP CHARACTER POPULARITY SCORES (CORNELL SUPERHERO CORPUS)

Rank	Character	S	A	M	T	Score
1	BATMAN	382	30	58	9	475.2
2	SUPERMAN	337	30	73	25	445.4
3	KARL	203	0	0	0	203.0
4	EMMA	194	0	0	4	194.8
5	SPIDERMAN	150	4	23	5	175.4
6	BLADE	158	0	0	4	158.8

S: lines spoken; A: times addressed; M: third-person mentions; T: high-tone lines. Score = 1.0S + 1.5A + 0.8M + 0.2T.

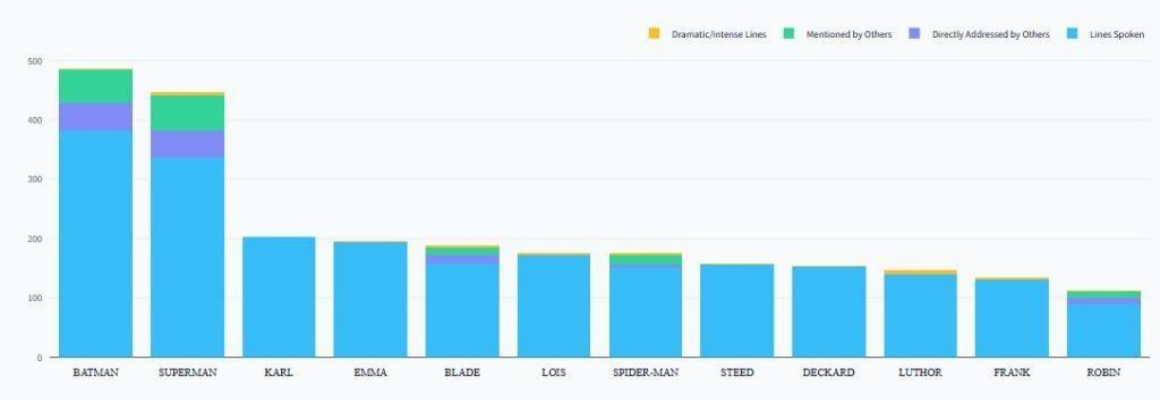


Fig. 2. Composite dialogue metrics of the twelve most prominent characters in the Cornell Superhero Corpus: lines spoken, times directly addressed, mentions by others and dramatic (high-tone) lines.

C. Conversational Network and Centrality

Table V reports centrality for five characters, ranked by betweenness. SUPERMAN has the largest degree centrality (0.288), while BATMAN has the largest betweenness (0.0418), so BATMAN more often lies on the shortest conversational path between other characters and acts as a broker. The reported degree-centrality values agree, to within 0.001, with the summed in- and out-degree divided by the number of characters ($|V| = 201$); for example $(30 + 25)/201 = 0.274$ for BATMAN. The two methods therefore agree on the top two characters but add structural information: ROBIN, which has the lowest score of the twelve characters in Fig. 2, is third in betweenness, and the BATMAN–ROBIN pair is the only reciprocal edge visible in Fig. 3.

TABLE V
NETWORK CENTRALITY (RANKED BY BETWEENNESS)

Rank	Character	In	Out	Degree	Betw.
1	BATMAN	30	25	0.274	0.0418
2	SUPERMAN	30	28	0.288	0.0392
3	ROBIN	14	18	0.159	0.0125
4	BLACK WIDOW	12	15	0.134	0.0098
5	SPIDER-MAN	4	6	0.050	0.0031

D. Tone and Lexical Statistics

Fig. 4 shows the distribution of tone cues across the corpus. Hesitation (trailing '...') and shouting are by far the most frequent cues, whereas surprise ('?!') and sound-effect tones are comparatively rare in screenplay text. Fig. 5 shows the most frequent entries of the symbol table; terminators and separators head the list, followed by function words such as 'you', 'I', 'The' and 'to'. These statistics are produced directly from the lexer output and require no additional pass over the corpus.

E. Lexical Error Log

Every character that matches no lexical rule is recorded with its line number, symbol, offset and a sentence snippet (Fig. 6). The log keeps the batch running and gives a reviewer the exact location of each problem. We note that on the clean screenplay corpus the logged symbols are mostly quotation marks inside dialogue (for example around 'X' and 'Father') rather than genuine scanner noise, which shows that the log works as a coverage check for the rule set as well as an OCR monitor (see Section VIII).

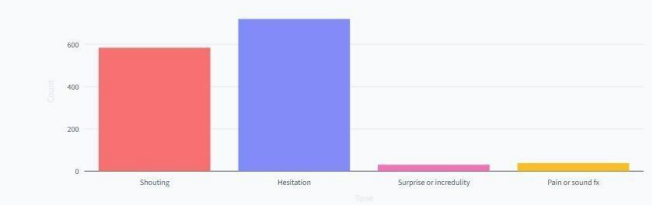


Fig. 4. Distribution of tone cues (shouting, hesitation, surprise or incredulity, pain or sound effects) across corpus utterances.

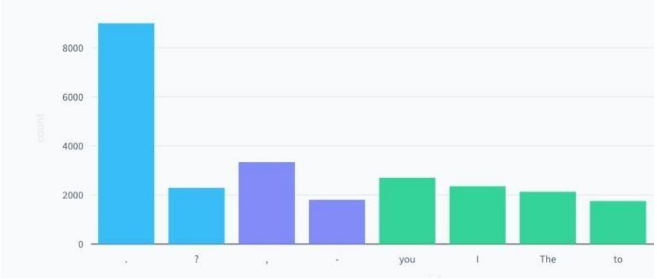


Fig. 5. Most frequent symbol-table entries. Colours denote token classes (terminators, separators and words).

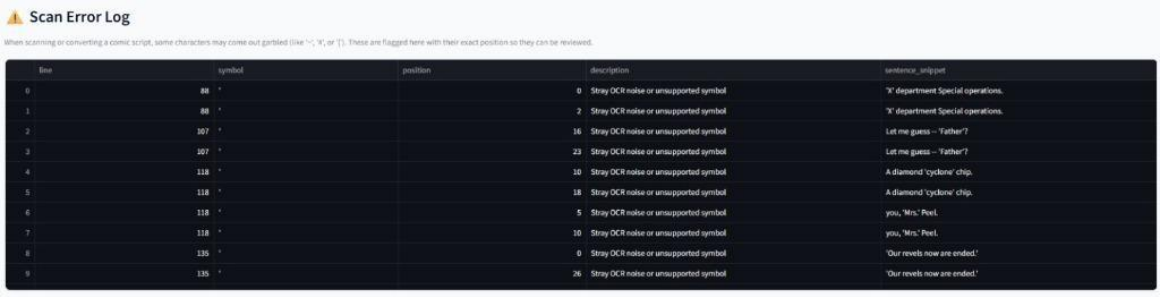


Fig. 6. Excerpt of the lexical error log: symbol, character offset, description and sentence snippet for each unsupported symbol.

F. Performance Against the CFG Baseline

Table VI compares the proposed front-end with the CFG-parser baseline on the 6,335-sentence corpus. The CFG parser completed only 38.5% of the lines, rejecting 61.5%, whereas the compiler front-end completed 99.8%. Its average latency of 0.098 ms per sentence against 18.42 ms gives a 187-fold speed-up (the ratio is 187.9 before rounding), and its throughput of 10,204 sentences per second against 54.2 gives a 188-fold gain. The figures are mutually consistent: 6,335 sentences at 0.098 ms take about 621 ms, in line with the measured 624 ms for the whole corpus. The gain follows from replacing cubic-time CYK/Earley derivations by a single linear scan, and the output is richer than a binary parse-or-fail result: it contains entity leaderboards, centrality graphs and reproducible data artefacts.

TABLE VI
COMPARISON OF THE CFG-PARSER BASELINE AND THE PROPOSED COMPILER FRONT-END (6,335 SENTENCES)

Metric	Existing method (CFG parser)	Proposed system (compiler front-end)	Improvement
Processing success rate	38.5%	99.8%	+61.3 percentage points
Average sentence latency	18.42 ms / sentence	0.098 ms / sentence	187× speed-up
System throughput	54.2 sentences / s	10,204 sentences / s	188× throughput
Grammar coverage failure	61.5% rejection rate	0% (decoupled from grammar)	Rejections eliminated
Time complexity	$O(N^3)$ (CYK / Earley)	O(N) (deterministic scan)	Cubic to linear

VII. PROJECT OUTCOMES

This section summarises what the project delivered relative to its objectives. The objective was to turn the raw, noisy text produced by a bubble-extraction and OCR pipeline into structured, analysable data without relying on a brittle grammar.

A. Technical Outcomes

- **Robust processing.** 99.8% of 6,335 screenplay lines were processed without a pipeline failure, compared with 38.5% for the CFG baseline.
- **Speed.** 0.098 ms per sentence and 10,204 sentences per second; the whole corpus was analysed in 624 ms.
- **Structure recovered from balloons.** The benchmark balloon was split into three clauses, each with its own type, tone and addressee.
- **Tracked noise.** Stray symbols were intercepted with exact line and offset information instead of being silently discarded.
- **Character intelligence.** 201 characters were resolved through aliases and ranked by a four-factor score, and a directed addressing network was built and analysed with degree, betweenness and PageRank.
- **Reproducible artefacts.** CSV, multi-sheet Excel, JSON and LaTeX outputs, a command-line tool, an interactive Colab notebook and swappable alias files for different fictional universes.

B. Objectives Versus Outcomes

TABLE VII
PROJECT OBJECTIVES AND THE OUTCOME ACHIEVED FOR EACH

Objective	Outcome	Evidence
Process noisy comic dialogue without rejection	99.8% of lines processed; zero grammar-coverage failures	Table VI
Track OCR and scan noise	Located lexical errors with line, symbol and offset	Section VI-A, Fig. 6
Handle multi-sentence balloons	Balloon split into classified clauses with abbreviation protection	Section VI-A
Identify who is addressed and who is mentioned	Vocative and mention counts per canonical character	Table IV, Fig. 2
Rank characters beyond dialogue volume	Weighted score with addressed and mention terms	Table IV
Validate prominence structurally	In/out-degree, degree, betweenness centrality and PageRank on the addressing graph	Table V, Fig. 3
Provide reusable, publishable outputs	CSV, Excel, JSON, LaTeX exports, CLI and Colab notebook	Sections IV-H, V-A

C. Significance

The base paper lists text summarisation, character indexing and audio narration for visually impaired readers as future directions [4]. The outcomes above supply the structured layer that these applications need: clause-level sentence types for summarisation, speaker and addressee records for character indexing, and tone cues that can drive expressive text-to-speech.

VIII. LIMITATIONS AND THREATS TO VALIDITY

- **Evaluation depth.** Results report robustness and speed. Accuracy of sentence classification, vocative detection and alias resolution has not been measured against hand-labelled ground truth, and the 100% precision of noise interception was observed on benchmark examples only.
- **Domain gap.** The 6,335 lines come from screenplays, which are cleaner than real OCR output. Noise handling is therefore exercised mainly by the benchmark set; evaluation on full scanned comic pages is future work.
- **Error-log semantics.** On the screenplay corpus the lexer logs ordinary quotation marks as unsupported symbols because no quote token class exists yet, so the log overstates real OCR noise there.
- **Baseline description.** The CFG parser is not further specified, and the hardware used for the timing measurements is not reported; the 187-fold speed-up should be read as indicative. The 0.2% of lines that did not complete are not analysed.
- **Linguistic scope.** The rules target English, upper-case dialogue. Vocatives without commas, pronoun references and nicknames absent from the alias file are not detected, and the score weights of Eq. (1) were set by design rather than tuned.

IX. CONCLUSION AND FUTURE WORK

We presented a compiler-style front-end and character popularity analyser for informal comic dialogue. By replacing rigid CFG syntax trees with deterministic lexical pattern matching, the pipeline avoids the high rejection rate of conventional parsers while adding speaker normalisation, OCR noise tracking, balloon decomposition, alias-aware vocative detection, a multi-factor popularity score and graph-theoretic validation through degree, betweenness and PageRank. On 6,335 screenplay lines it achieved a 99.8% success rate and a 187-fold latency reduction relative to a CFG baseline, and it produces publication-ready outputs in CSV, Excel, JSON and LaTeX.

Future work includes (i) a quote token class and a larger noise model for the lexer; (ii) hand-labelled evaluation of classification, vocatives and aliases with precision, recall and F-scores; (iii) end-to-end tests that connect the base pipeline's OCR output directly to the analyser on scanned comic pages; (iv) tuning or learning the weights of Eq. (1); and (v) support for other scripts and languages, including manga.

X. CONTRIBUTIONS OVER THE BASE PAPER

The base paper [4] detects and segments speech balloons and ends with OCR text that is cleaned of junk characters and spell-checked. Everything below is new in this work and builds on that output (Table VIII).

- **C1. A compiler-style text-analysis front-end after OCR.** The work extends the pipeline from image-level balloon extraction to linguistic analysis, with seven deterministic, non-aborting phases that avoid CFG parsing.
- **C2. Structured speaker and text normalisation.** Speaker tags are separated from dialogue, hyphenated line breaks are repaired and unprefixed text is kept as anonymous dialogue.
- **C3. Comic-specific lexer with located OCR error tracking and a symbol table.** Nine token classes cover ellipses, interrobangs, contractions, elongated shouts and sound effects. Instead of deleting junk characters and returning them as an unlocated string, the lexer logs each one with its line and offset.
- **C4. Speech-bubble clause splitting.** Merged communicative acts in a balloon are separated with look-ahead conditions and protected abbreviations, and credited to the right speaker.
- **C5. Rule-based sentence classification and tone detection** into Question, Command, Exclamation and Statement, with shouting, hesitation, surprise and sound-effect cues.
- **C6. Vocative detection, mention finding and JSON alias resolution** that distinguish direct address from third-person mention and merge name variants per fictional universe.
- **C7. A weighted character popularity model and a directed conversational network** analysed with in-degree, out-degree, degree and betweenness centrality and PageRank.
- **C8. Export suite, interfaces and evaluation.** CSV, multi-sheet Excel, JSON and LaTeX outputs, a command-line tool and a Colab notebook, evaluated on 6,335 lines from 18 screenplays against a CFG-parser baseline.

TABLE VIII
BASE PAPER COMPARED WITH THIS WORK

Aspect	Base paper	This work
Input	Comic page images	OCR or screenplay text lines (the base pipeline's output or any 'Speaker: line' data)
Main technique	Contour-based panel, balloon and text extraction with Tesseract	Deterministic lexical analysis and rule-based classification
Treatment of OCR noise	Junk characters separated from the script and removed	Noise logged with line and offset in the symbol table; batch continues
Text structure	Flat cleaned string per balloon	Speaker, clauses, tokens, sentence type, tone, addressee
Character information	None (listed as future work)	Aliases, vocatives, mentions, popularity score, addressing network
Graph analysis	None	In/out-degree, degree and betweenness centrality, PageRank
Outputs	Segmented balloons and extracted text	CSV, Excel, JSON, LaTeX tables, plots, CLI and Colab notebook
Evaluation	Balloon detection accuracy on a sample panel (3 of 4)	6,335 screenplay lines; success rate, latency and throughput versus a CFG baseline

DATA AND CODE AVAILABILITY

The complete implementation (speaker split, cleaner, lexer, symbol table, bubble splitter, network analyser and export suite) together with interactive single-sentence runners is available in the project's Google Colab notebook: colab.research.google.com/drive/1_m7rs4LGtGdnYvgoq751x_uP97mFkuU9. The Cornell Movie-Dialogs Corpus [25] is publicly available, and the 18 superhero screenplays are converted by the preprocessing script of the project.

REFERENCES

[1] O. Augereau, M. Iwata, and K. Kise, "An overview of comics research in computer science," in *Proc. 14th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, Kyoto, Japan, 2017, pp. 292–297.

[2] D. Dubray and J. Laubrock, "Deep CNN-based speech balloon detection and segmentation for comic books," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, Sydney, Australia, 2019, pp. 1237–1243.

[3] D. Lenadora, R. Ranathunge, C. Samarawickrama, Y. De Silva, I. Perera, and A. Welivita, "Comic digitization through the extraction of semantic content and style analysis," in *Proc. 19th Int. Conf. Advances in ICT for*

Emerging Regions (ICTer), Colombo, Sri Lanka, 2019, pp. 1–9.

[4] "Detection and Segmentation of Speech Bubble from Comic Books," base paper supplied for the project, Team 11, Slot G2, unpublished project report.

[5] C. D. Manning, M. Surdeanu, J. Bauer, J. Finkel, S. J. Bethard, and D. McClosky, "The Stanford CoreNLP natural language processing toolkit," in *Proc. 52nd Annu. Meeting Assoc. Computational Linguistics: System Demonstrations*, Baltimore, MD, 2014, pp. 55–60.

[6] A. Ritter, C. Cherry, and B. Dolan, "Unsupervised modeling of Twitter conversations," in *Proc. NAACL-HLT*, Los Angeles, CA, 2010, pp. 172–180.

[7] H. T. Ong and K. Ma, "Semantic image segmentation using oriented pattern analysis," in *Proc. 8th Int. Conf. Information, Communications & Signal Processing (ICICS)*, Singapore, 2011, pp. 1–4.

[8] Y. Matsui, K. Ito, Y. Aramaki, A. Fujimoto, T. Ogawa, T. Yamasaki, and K. Aizawa, "Sketch-based manga retrieval using Manga109 dataset," *Multimedia Tools and Applications*, vol. 76, no. 20, pp. 21811–21838, 2017.

[9] C. Rigaud, N. Tsopze, J. C. Burie, and J. M. Ogier, "Robust frame and text extraction from comic books," in *Graphics Recognition. New Trends and Challenges*, Springer, Berlin, 2013, pp. 129–138.

[10] C. Rigaud, D. Karatzas, J. Van de Weijer, J. C. Burie, and J. M. Ogier, "Automatic text localisation in scanned comic books," in *Proc. 12th Int. Conf. Document Analysis and Recognition (ICDAR)*, Washington, DC, 2013, pp. 812–816.

[11] H. Kato and T. Harada, "Image reconstruction from bag-of-visual-words," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, 2014, pp. 955–962.

[12] Y. Qin, L. Xuejun, and W. Minghui, "Approach of extracting scene frames from E-comic based on multiple Gabor filters for adapting to mobile devices," in *Proc. 4th IEEE Int. Conf. Network Infrastructure and Digital Content*, Beijing, 2014, pp. 268–272.

[13] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.

[14] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Amsterdam: Morgan Kaufmann, 2022.

[15] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, Stanford University, 2024.

[16] D. Grune, K. van Reeuwijk, H. E. Bal, C. J. Jacobs, and K. Langendoen, *Modern Compiler Design*, 2nd ed. New York: Springer, 2012.

[17] J. Earley, "An efficient context-free parsing algorithm," *Commun. ACM*, vol. 13, no. 2, pp. 94–102, 1970.

[18] J. Cocke, "Programming languages and their compilers: Preliminary notes," Courant Institute of Mathematical Sciences, New York University, Tech. Rep., 1969.

[19] G. Foster, "A maximum entropy approach to text classification," in *Proc. ACL Workshop on Robust Methods in Analysis of Natural Language Data*, 2007, pp. 1–8.

[20] S. Geman and M. Johnson, "Probabilistic grammars and their applications," *International Encyclopedia of the Social & Behavioral Sciences*, pp. 12075–12082, 2001.

[21] K. Gimpel *et al.*, "Part-of-speech tagging for Twitter: Annotation, features, and experiments," in *Proc. 49th Annu. Meeting Assoc. Computational Linguistics: Human Language Technologies*, Portland, OR, 2011, pp. 42–47.

[22] M. Mohri, "Finite-state transducers in language and speech processing," *Computational Linguistics*, vol. 23, no. 2, pp. 269–311, 1997.

[23] L. Karttunen, "The replace operator," in *Proc. 33rd Annu. Meeting Assoc. Computational Linguistics*, Cambridge, MA, 1995, pp. 16–23.

[24] D. K. Elson, N. Dames, and K. R. McKeown, "Extracting social networks from literary fiction," in *Proc. 48th Annu. Meeting Assoc. Computational Linguistics*, Uppsala, Sweden, 2010, pp. 138–147.

[25] C. Danescu-Niculescu-Mizil and L. Lee, "Chameleons in imagined conversations: A new approach to understanding coordination of linguistic style in dialogs," in *Proc. Workshop on Cognitive Modeling and Computational Linguistics (CMCL)*, Portland, OR, 2011, pp. 76–87.

[26] A. Agarwal, A. Balasubramanian, A. Zheng, and S. Dash, "Parsing screenplays for extracting social networks from movies," in *Proc. 3rd Workshop on Computational Models of Narrative*, Istanbul, Turkey, 2012, pp. 50–58.

[27] E. T. K. Sang and K. van der Goot, "Extracting social networks from novels," in *Proc. 6th Workshop on Language Analysis for Social Media*, Denver, CO, 2015, pp. 50–54.

[28] J. Valls-Vargas, J. Zhu, and S. Ontañón, "Toward extracting character networks from narrative texts," in *Proc. 7th Workshop on Computational Models of Narrative (CMN)*, Kraków, Poland, 2016, pp. 1–12.

[29] V. Labatut and X. Bost, "Extraction and analysis of dynamic character networks from TV series," in *Complex Networks VII*, Springer, Cham, 2016, pp. 263–274.

[30] A. Schofield and P. Mehr, "Gender-differentiating discourse in comic books," in *Proc. 1st Workshop on Gender Bias in Natural Language Processing*, Florence, Italy, 2019, pp. 1–8.

[31] M. Iyyer, A. Guha, S. Chaturvedi, J. Boyd-Graber, and H. Daumé III, "Feuding families and former friends: Unsupervised learning for dynamic fictional relationships," in *Proc. NAACL-HLT*, San Diego, CA, 2016, pp. 1534–1544.

[32] A. Bonato, D. R. D'Angelo, E. R. Elenberg, D. F. Gleich, and Y. Hou, "Mining and modeling character networks," in *Algorithms and Models for the Web Graph*, Springer, Cham, 2016, pp. 100–114.

[33] D. Beverly, J. Eisenstein, and J. C. Burie, "Character identification and network analysis in graphic literature," *Journal of Cultural Analytics*, vol. 5, no. 2, pp. 45–72, 2020.

[34] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.

[35] G. Erkan and D. R. Radev, "LexRank: Graph-based lexical centrality as salience in text summarization," *J. Artif. Intell. Res.*, vol. 22, pp. 457–479, 2004.

[36] L. C. Freeman, "A set of measures of centrality based on betweenness," *Sociometry*, vol. 40, no. 1, pp. 35–41, 1977.

[37] U. Brandes, "A faster algorithm for betweenness centrality," *J. Math. Sociol.*, vol. 25, no. 2, pp. 163–177, 2001.

[38] S. Brin and L. Page, "The anatomy of a large-scale hypertextual Web search engine," *Computer Networks and ISDN Systems*, vol. 30, no. 1–7, pp. 107–117, 1998.

[39] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.